

Interconnection Structures

Condensed Study Notes

Generated: 2026-04-28 11:57 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

This initial section serves as the title page for the 'COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)'. No further instructional content or definitions were provided in this segment.

Interconnection Structures

This section introduces fundamental concepts related to how different parts of a computer system communicate and how instructions are structured and managed.

Bus Structures

A **bus** is a shared communication pathway that connects various components within a computer, allowing them to exchange data.

- **Single Bus:** All components (like the CPU, memory, and input/output devices) share one common bus for all data transfers. This simplifies design but means only one transfer can occur at a time, potentially creating a bottleneck.
- **Multiple Bus:** The system uses several distinct buses. This can improve performance by allowing more concurrent data transfers between different sets of components.

Instruction and Instruction Sequencing

This topic broadly covers the process by which a computer's central processing unit (CPU) fetches, decodes, and executes instructions in a specific, ordered sequence to perform tasks.

Instruction Set Architecture (ISA)

The **Instruction Set Architecture (ISA)** defines the set of instructions a particular processor can understand and execute. It acts as the interface between software and hardware.

- **CISC (Complex Instruction Set Computer):** An ISA design where single instructions can perform multiple complex operations, often combining memory access, arithmetic, and register manipulation in one step.
- **RISC (Reduced Instruction Set Computer):** An ISA design characterized by a smaller, simpler, and highly optimized set of instructions, where each instruction typically performs only one basic operation.

Other Key Concepts

This section also introduces:

- **Register Transfer Notation (RTN):** A formal language used to describe the micro-operations within a computer, specifically detailing how data is transferred between registers.
- **Instruction Types:** Categories of instructions that a CPU can execute, such as those for data movement, arithmetic calculations, logical operations, or controlling program flow.

Interconnection structures

Every computer system is built upon three fundamental modules: the **Processor**, **Memory**, and **I/O** (Input/Output).

These core modules must communicate continuously with each other to operate. To facilitate this essential communication, dedicated pathways are required.

The **interconnection structure** is the collection of these pathways that link the various modules within a computer. The specific design of this structure is dictated by the types of data and control exchanges that need to occur between the modules.

(Recall that last week we discussed bus structures as a common type of interconnection.)

I/O module

The **I/O module** acts as the interface between the computer system and external devices, enabling communication through basic read and write operations. The overall interconnection structure of the computer, as discussed previously, must support various types of data transfers involving this module.

Key Data Transfers:

- **Processor ↔ Memory:** The processor reads instructions or data from memory, and writes data back to memory.
- **Processor ↔ I/O:** The processor reads data from an I/O device (via the I/O module) or sends data to an I/O device.
- **I/O ↔ Memory (Direct Memory Access - DMA):** The I/O module can directly exchange data with memory without involving the processor. This dedicated pathway is known as **Direct Memory Access (DMA)**, significantly speeding up data transfers by bypassing the CPU.

I/O Module Capabilities:

- Can control more than one external device.
- Each connection point to an external device is called a port, which has a unique address (e.g., 0, 1, M-1).
- Includes external data paths for input and output.
- Can send interrupt signals to the processor, indicating an event or request.

From an internal perspective, the I/O module's function of exchanging data is similar to how memory operates.

Memory

Memory serves as the storage location for data, with operations precisely controlled.

Key aspects:

- **Address:** A unique numerical identifier that specifies the exact location for a memory operation.

- **Read/Write Control Signals:** These signals indicate the type of operation: either retrieving data (**read**) or storing data (**write**).
- **Word:** The basic unit of data that can be read from or written into memory. Each word is assigned a unique numerical address (e.g., 0, 1, ..., N-1).

*A typical memory module is structured to contain N * words, all of which are of equal length.*

Complex Instruction Set Architecture (CISC)

Complex Instruction Set Architecture (CISC) is one approach to increasing CPU performance. The core idea behind CISC is to make the hardware complex enough to allow a single instruction to perform multiple low-level operations.

This means a single CISC instruction can handle tasks like:

- Loading data
- Performing an operation (e.g., evaluating)
- Storing the result

For example, a multiplication command in CISC can execute all these steps within a single instruction, making it 'complex'.

Instruction set Architecture: CISC, RISC

This section introduces two primary approaches to designing a computer's Instruction Set Architecture (ISA): Complex Instruction Set Computing (CISC) and Reduced Instruction Set Computing (RISC).

As previously covered, **Complex Instruction Set Architecture (CISC)** aims to boost CPU performance by using single, intricate instructions. These instructions are capable of executing multiple operations, such as loading data, performing calculations, and storing results, all within one instruction.

While **Reduced Instruction Set Architecture (RISC)** is named as a counterpart, its specific details are not provided in this material.

Reduced Instruction Set Architecture (RISC)

Reduced Instruction Set Architecture (RISC) is an approach focused on simplifying computer hardware. This simplification is achieved by using an instruction set composed of only a few basic steps.

These simple instructions perform fundamental operations like loading, evaluating, and storing data. For instance, a 'load' command specifically loads data, and a 'store' command stores data. This contrasts with Complex Instruction Set Architecture (CISC), which uses single, complex instructions to perform multiple operations.

The “best” way to design a CPU has been a subject of debate

The design of a CPU often sparks debate, especially regarding how instructions should be structured and executed. This fundamental choice—between **CISC** and **RISC**—is a key way to categorize different computer architectures.

At the heart of the debate is this question: Should low-level commands be long and powerful, performing complex tasks with fewer instructions (CISC), or should they be shorter and simpler, requiring more instructions for the same task but potentially leading to fewer cycles per instruction and more efficient code (RISC)?

Key characteristics of these two CPU design philosophies:

- **CISC (Complex Instruction Set Computer)**: Uses a large set of complex machine language instructions. A single CISC instruction can execute multiple operations, meaning fewer individual instructions are needed to perform a complex task.
- **RISC (Reduced Instruction Set Computer)**: Employs a reduced set of simpler, fundamental instructions. While a complex task might require more individual RISC instructions, this approach can allow for fewer cycles per instruction and contribute to more efficient code overall.

Characteristic of RISC

Reduced Instruction Set Computing (RISC) architectures are defined by several key characteristics that prioritize simplicity and speed:

*

Simpler Instructions & Execution:

- RISC instructions are inherently simpler, leading to straightforward instruction decoding (interpreting what an instruction means).
- These instructions are compact, typically fitting within a single word (the CPU's standard fixed-size data unit).
- Each instruction is designed to execute in just one clock cycle, the basic unit of time for CPU operations, which significantly speeds up processing.

*

Registers & Data Handling:

- RISC CPUs typically feature more general-purpose registers. These are fast storage locations directly within the CPU, used to hold data or addresses, reducing the need to access slower main memory frequently.
- They support fewer data types (e.g., integers, floating-point numbers) directly, which simplifies the CPU's design.

*

Addressing & Pipelining:

- RISC uses simple addressing modes, which are straightforward methods for the CPU to locate the data (operands) an instruction needs.
- The simplicity and consistent execution time of RISC instructions enable efficient **pipelining**. Pipelining is an optimization technique where multiple instructions are processed in overlapping stages,

much like an assembly line, allowing the CPU to complete instructions at a faster overall rate.

Characteristic of CISC

CISC (Complex Instruction Set Architecture) is characterized by a design philosophy that favors powerful, single instructions capable of performing multiple operations.

Key characteristics:

- **Complex Instructions:** CISC instructions are inherently complex, leading to more intricate decoding processes.
- **Instruction Size:** Instructions are often larger than a single 'word' (a fixed-size unit of data memory can store).
- **Execution Time:** Instructions may take more than one clock cycle to complete execution.
- **Registers & Memory:** CISC architectures typically use fewer general-purpose registers because many operations can be performed directly in memory.
- **Addressing Modes:** They support complex addressing modes, which define how the location of an operand (the data an instruction operates on) is specified.
- **Data Types:** CISC supports a wider variety of data types.

CISC vs. RISC Operation (Example):

Consider adding two numbers:

- **CISC Approach:** A single command, like ADD, can perform the entire task (loading, operating, and storing the result).
- **RISC Approach (for comparison):** A programmer would write multiple distinct commands: first to LOAD data into registers, then apply a suitable OPERATOR, and finally STORE the result. This means RISC programs tend to be longer and require more memory to store, but they achieve this with less complex commands, leading to fewer transistors.

CISC vs. RISC: Design Trade-offs

The debate over which design approach—CISC or RISC—is "better" for computer architecture has no definitive answer; each involves different performance trade-offs.

Core Trade-offs:

- **CISC (Complex Instruction Set Architecture):** Designed to minimize the total number of instructions within a program. However, this often means each complex instruction requires more execution cycles.
- **RISC (Reduced Instruction Set Architecture):** Aims to reduce the number of cycles per instruction, making individual instructions execute very quickly. This typically results in a higher total number of instructions needed to complete a program.

Historical Context:

CISC architecture emerged when programming in assembly language was common. Complex instructions were preferred to reduce the tedious and error-prone nature of assembly coding. With the shift towards high-level languages, the need for intricate assembly instructions lessened, and RISC architecture

became more prevalent.

Conclusion:

Both CISC and RISC designs have unique advantages and disadvantages. For instance, x86 CPUs commonly use CISC, while ARM processors (found in many mobile devices), PowerPC, and Sparc utilize RISC.

Difference

This section outlines the key differences between Reduced Instruction Set Computer (RISC) and Complex Instruction Set Computer (CISC) architectures, building on their core philosophies.

RISC vs. CISC: Key Differences

- **Design Emphasis:**

- **RISC:** Focuses on software, implying that complexity is often managed through software optimization (e.g., compilers).
- **CISC:** Focuses on hardware, implying that complex instruction execution is handled directly by the hardware.

- **Instruction Size & Execution:**

- **RISC:** Utilizes fixed-sized instructions, which typically execute in a single clock cycle.
- **CISC:** Employs variable-sized instructions, often requiring more than one clock cycle for execution.

- **Register Usage:**

- **RISC:** Requires a greater number of registers. Transistors in RISC architectures are primarily allocated for these abundant registers.
- **CISC:** Requires fewer registers. Transistors in CISC architectures are used more for storing complex instruction logic rather than numerous registers.

- **Operation Types:**

- **RISC:** Can perform only Register-to-Register operations, moving data between registers.
- **CISC:** Can perform a wider range of operations, including Register-to-Register, Register-to-Memory (data between register and memory), or Memory-to-Memory (data directly between memory locations).

- **Code Size:**

- **RISC:** Tends to result in a larger code size, as more simple instructions are often needed to complete a task.
- **CISC:** Tends to result in a smaller code size, as complex instructions can achieve more in a single step.

Instruction and Instruction Types

An **instruction set** defines the interface between software modules and the underlying hardware; it dictates what the hardware will do under specific circumstances.

Beyond the basic RISC/CISC characterization, computers can be classified by several characteristics of their instruction sets. These include:

- Addressing modes
- Numbers of operands
- Types of operations supported
- Fixed versus variable instruction length

Word length

[Content could not be generated for this section.]

INSTRUCTION SEQUENCING

[Content could not be generated for this section.]

References

[Content could not be generated for this section.]